

Cheat sheet

```

initially,  $\epsilon$ -closure( $s_0$ ) is the only state in  $Dstates$ , and it is unmarked;
while ( there is an unmarked state  $T$  in  $Dstates$  ) {
    mark  $T$ ;
    for ( each input symbol  $a$  ) {
         $U = \epsilon$ -closure(move( $T, a$ ));
        if (  $U$  is not in  $Dstates$  )
            add  $U$  as an unmarked state to  $Dstates$ ;
         $Dtran[T, a] = U$ ;
    }
}

```

Figure 3.32: The subset construction

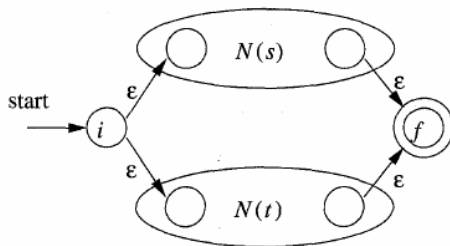


Figure 3.40: NFA for the union of two regular expressions

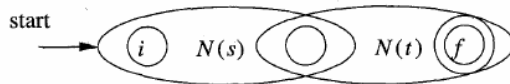


Figure 3.41: NFA for the concatenation of two regular expressions

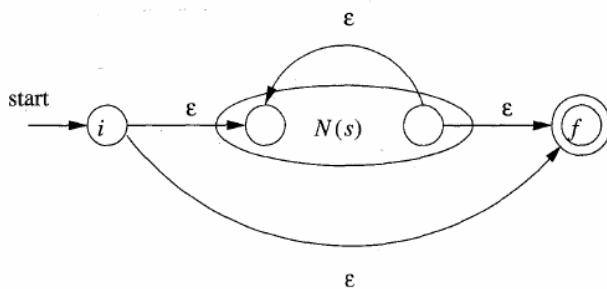


Figure 3.42: NFA for the closure of a regular expression

METHOD:

1. Start with an initial partition Π with two groups, F and $S - F$, the accepting and nonaccepting states of D .
2. Apply the procedure of Fig. 3.64 to construct a new partition Π_{new} .

```

initially, let  $\Pi_{\text{new}} = \Pi$ ;
for ( each group  $G$  of  $\Pi$  ) {
    partition  $G$  into subgroups such that two states  $s$  and  $t$ 
    are in the same subgroup if and only if for all
    input symbols  $a$ , states  $s$  and  $t$  have transitions on  $a$ 
    to states in the same group of  $\Pi$ ;
    /* at worst, a state will be in a subgroup by itself */
    replace  $G$  in  $\Pi_{\text{new}}$  by the set of all subgroups formed;
}

```

Figure 3.64: Construction of Π_{new}

3. If $\Pi_{\text{new}} = \Pi$, let $\Pi_{\text{final}} = \Pi$ and continue with step (4). Otherwise, repeat step (2) with Π_{new} in place of Π .
4. Choose one state in each group of Π_{final} as the *representative* for that group. The representatives will be the states of the minimum-state DFA D' . The other components of D' are constructed as follows:

- 1) arrange the nonterminals in some order $A_1, A_2, \dots, A_n$.
- 2) for (each i from 1 to n) {
- 3) for (each j from 1 to $i - 1$) {
- 4) replace each production of the form $A_i \rightarrow A_j \gamma$ by the productions $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$, where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ are all current A_j -productions
- 5) }
- 6) eliminate the immediate left recursion among the A_i -productions
- 7) }

Figure 4.11: Algorithm to eliminate left recursion from a grammar

METHOD: For each nonterminal A , find the longest prefix α common to two or more of its alternatives. If $\alpha \neq \epsilon$ — i.e., there is a nontrivial common prefix — replace all of the A -productions $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$, where γ represents all alternatives that do not begin with α , by

$$\begin{aligned}
 A &\rightarrow \alpha A' \mid \gamma \\
 A' &\rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n
 \end{aligned}$$

Define $\text{FIRST}(\alpha)$, where α is any string of grammar symbols, to be the set of terminals that begin strings derived from α . If $\alpha \xRightarrow{*} \epsilon$, then ϵ is also in $\text{FIRST}(\alpha)$. For example, in Fig. 4.15, $A \xRightarrow{*} c\gamma$, so c is in $\text{FIRST}(A)$.

Define $\text{FOLLOW}(A)$, for nonterminal A , to be the set of terminals a that can appear immediately to the right of A in some sentential form; that is, the set of terminals a such that there exists a derivation of the form $S \xRightarrow{*} \alpha A a \beta$, for some α and β , as in Fig. 4.15. Note that there may have been symbols between